

CLAUDE MODS / STARTER KIT

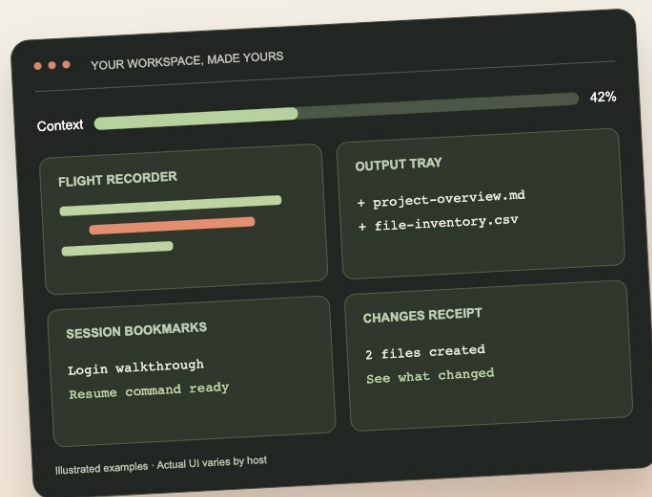
Your Claude. Your rules.

CLAUDE MODS / STARTER KIT

PROMPT ADVISERS

Your Claude. Your rules.

Ten mods. Clear guides.
Prompts to build your own.



INSTALL IT • TRY IT • SWITCH IT OFF

A practical guide to ten mods, the commands that control them, and the prompts to build your own.

Prompt Advisers · Release 1.0.0 · October 2, 2026

Community source for Claude Code. Tested with CLI 2.1.287 on macOS. Illustrated UI examples, not screenshots of a running session. No model subscription or API credit included.

Choose your first win

Want to...	Start with...
See something instantly	Terminal Pet: <code>/pet party</code>
Understand conversation fullness	Context Meter: <code>automatic</code>
See which files Claude reads	Repo Heatmap: <code>/heatmap open</code>
See helpers working	Flight Recorder: <code>/timeline open</code>
Choose a lighter helper model	Model Router: <code>/router</code>
Find newly created files	Output Tray: <code>/tray show</code>
See what changed after a task	Changes Receipt: <code>/receipt last</code>
Return to an earlier conversation	Session Bookmarks: <code>/bm list</code>
Brief a fresh conversation	Auto Handoff: <code>/autohandoff now</code>
Change the screen's visual style	Coral Skin: <code>/skin on</code>

Read Start Here first. Each mod then has a use guide followed by its complete creation prompt. Code is also provided in separate copy-ready files in the ZIP; PDF line wrapping is for reading.

Start here: your first mod

No development experience required. Start with the pet, then add what you need.

1. Open the guide without installing anything

Open **VIEWER-GUIDE.html** in your browser. It works offline, has every command and creation prompt, and includes a search box. Or open **CLAUDE-MODS-VIEWER-GUIDE.pdf**. Neither runs or installs a mod.

2. Check the app

These are community mods for **Claude Code**, tested with CLI version **2.1.287** on macOS on October 2, 2026. They are not browser extensions or add-ons for ordinary Claude chat. You need an installed, signed-in Claude Code with mods support and a supported plan or API account. Check official setup:

<https://code.claude.com/docs/en/quickstart>

Open Terminal and run:

```
claude --version
```

If that command is missing, follow official setup first. You also need Git for marketplace installation and some project features. The helper scripts use Bash and Python 3. Check with `git --version` and `python3 --version`. Windows users: the shell workflow assumes WSL; Windows and Linux have not been live-tested. Output Tray's Open/Reveal buttons currently require macOS.

3. Try one mod without a lasting install

Unzip this kit, open Terminal, type `cd` (including the space), drag the extracted **claude-mods-starter-kit** folder into Terminal, and press Return. Then run:

```
bash scripts/try.sh terminal-pet
```

The script copies a tiny sample project into a new temporary folder and starts Claude Code there with only this kit's Pet plugin requested. Existing user plugins/settings may still load. It does not change your normal plugin settings or overwrite a prior demo. Review any normal folder-trust prompt.

In the Claude prompt, enter:

```
/pet party
```

You should see the pet celebrate. Next use `/pet off` to hide it. Type `/exit` to leave the temporary session. Temporary files stay in the printed folder until you choose to remove them or your system clears them.

4. Install for all your projects

Run these in Terminal:

```
claude plugin marketplace add promptadvisers/claude-mods-starter-kit --scope user
claude plugin install terminal-pet@claude-mods-kit --scope user
```

Then start a fresh Claude Code session. For a local ZIP-only install, from the extracted kit root use `claude plugin marketplace add . --scope user` instead. Keep that folder in a stable location while using the local marketplace.

5. Prefer one project?

Open Terminal in that project's folder. Add the marketplace and install using project scope:

```
claude plugin marketplace add promptadvisers/claude-mods-starter-kit --scope project
claude plugin install terminal-pet@claude-mods-kit --scope project
```

Project settings can be shared with collaborators; review them before committing. A private choice for only this project uses `--scope local` instead. Do not install every scope just in case: pick the scope you actually want.

6. Desktop is a separate running session

Choose the same project folder in the **Claude Code** area of the desktop app and start a fresh code session after installation. Fully quit and reopen if it still has old settings. A `claude --plugin-dir ... terminal` launch does not add a mod to an already open desktop chat. A normal Claude conversation is not the same thing as a Claude Code project session. Support varies by version and surface; a mock UI test is not proof of a live desktop result.

7. Choose your next mod

Read the guide or README catalog. Try the Context Meter next; then Heatmap, Timeline, Output Tray and Receipt. Router changes model selection. Handoff writes files and can make a model call. Choose those deliberately.

8. Switch this kit off

From the extracted kit root:

```
bash scripts/manage.sh disable user all
```

For a project install, run `bash /path/to/kit/scripts/manage.sh disable project all` from that project folder. Replace `/path/to/kit` with the actual extracted folder path; dragging the script into Terminal is an easy way to get it. Repeat with `local` if that is where you installed it. Then close old sessions. This targets the ten plugins in this kit, not unrelated plugins.

Next: [Scopes and troubleshooting](#) · [Build your own](#) · [Data and removal](#)

Where to type, scopes, and quick fixes

Three kinds of instruction

A **terminal command** tells your computer to install or start something. It starts with `claude`, `bash`, `git` or `cd`. Paste it in Terminal at the shell prompt, not into the chat.

A **slash command** tells a loaded mod what to do. Paste `/pet party` or `/tray list` into the Claude Code message box, one command at a time.

A **prompt** is a normal sentence asking Claude to work. Some mods react automatically; a few register a tool so Claude can control them by name. English is not a universal replacement for installing or invoking a mod.

The scope is where the switch lives

- **User:** your default across projects. Stored under your Claude user settings.
- **Project:** this folder's shared settings, usually `.claude/settings.json`. Teammates can see these if committed; they still need compatible software and permissions.
- **Local:** your private choice for this project, usually `.claude/settings.local.json`.
- **Temporary:** `--plugin-dir` loads source into the process you start. Closing it ends that load. It does not disable other installed plugins.

A narrower setting can override a wider one. Organization policy can also restrict plugins. If behavior disagrees with your user switch, inspect project/local settings using Claude's plugin controls. Do not delete whole settings files to solve one plugin problem.

Install, enable and load are different

Install makes a plugin available in a scope. Enable turns on an installed plugin in that scope. A fresh session loads it. If you previously disabled a plugin, re-enable it explicitly:

```
claude plugin enable terminal-pet@claude-mods-kit --scope user
```

Use the same marketplace identifier you actually installed. The video creator's private marketplace was named `my-mods`; this public kit is named `claude-mods-kit`.

It says the slash command does not exist

Check that you are in Claude Code, not a shell. Check the plugin is installed/enabled for this folder, then start a fresh session. `claude plugin list` shows installed plugins. The meter has no slash command. Names are `/timeline`, `/bm` and `/autohandoff`, not the plugin folder names.

It works in Terminal but not desktop

A temporary terminal launch only loads that process. Install at user scope or at the exact project folder selected in desktop. Start a fresh Claude Code project session. Check the app's own version and mod support; terminal and desktop can differ. Test one mod before combining them.

A panel is empty or missing

Widen the window and close another panel. Run the command again, then perform a new task in the same session. Heatmap tracks file activity; Tray tracks new files; Receipt needs a completed tracked turn. Work from another tab is not guaranteed to appear. `/tray list` and `/receipt last` give text fallbacks.

Router shows no savings

A helper must actually run. Its model must be recognized and more expensive than the target. The router may correctly leave it unchanged. Estimates are not invoices or a promise of equal answer quality. Keep default subagents mode; step switching can cost more due to cache loss.

Resume did not switch the chat

Bookmarks copy a command. Paste that command into a new terminal to reopen the original conversation. Handoff is different: open a fresh chat in the same project and ask it to read the summary. Neither is a one-click desktop chat switch.

Try all ten deliberately

From the kit root, `bash scripts/try.sh` all requests all ten for one terminal session. Several panes share limited room. Open one at a time. Router will affect supported helpers; Handoff defaults to automatic saves near 85% context. This combined workflow has not been live-certified across surfaces. For persistent installation, add the marketplace first, then use `bash scripts/manage.sh install user all`.

Build a mod from a plain-English idea

Imagine a car from the factory. You can add mood lighting, a fuel gauge, a dashcam or a place to keep your things. Claude Code mods add small changes like these to your work screen and workflow.

A mod is the change you want. A plugin is the package that carries it into Claude Code.

What happens under the hood?

1. Something happens: Claude starts a task, reads a file, finishes a tool, measures context, or draws the screen. 2. Claude Code sends an event: a small message describing that thing. 3. The mod's code handles that event: it can update a count, draw a panel, save a summary, or, for some hooks, change how work proceeds. 4. The normal work continues. A well-built mod preserves permission prompts and the next handler in the chain.

A prompt asks Claude to write the mod. After that, code reacts to events. The pet does not need a new AI answer for every animation frame. Some features, such as writing a handoff, deliberately make another model call.

A five-part brief

Tell Claude: what event to notice, what result to show, where it should appear, how you turn it off, and how you will test it. "Make it look cool" is not enough to define reliable behavior.

Use the complete prompts

The `prompts/` folder contains one full specification per mod. Paste the whole file into Claude Code in a new project. Add your own colors or style. The prompts require the assistant to inspect your current host APIs, validate the plugin, test failure cases and explain untested behavior. A prompt cannot guarantee a perfect build.

A tiny editable starter

Copy `templates/starter-mod` to a new folder. It contains a minimal command that counts successful Read events and prints the count. Rename its manifest name, hook state namespace and command before building a separate plugin. It is a teaching template, not one of the ten featured mods.

Typical files:

<code>my-plugin/</code>	
<code> .claude-plugin/plugin.json</code>	package name and version
<code> hooks/hooks.json</code>	which module to load
<code> hooks/starter.mjs</code>	event handlers

`tests/starter.test.ts``behavior checks`

The `.claude-plugin/types` directory may be generated by your local host. Its API definitions depend on your installed version; do not copy someone else's generated types as a permanent API contract. The bundled `types/index.d.ts` files describe each mod's own state.

Make the first version smaller

Build one event and one visible response. Test it in a throwaway folder with `--plugin-dir`. Ask for a failed-tool test and an off switch. Only then add a panel, saved preferences or more events. Never test destructive actions on your real work to make a demo look exciting.

Review before sharing

Remove settings, chat transcripts, handoffs, bookmarks, private paths, keys, screenshots with personal data and generated caches. Include the source, its tests, instructions and a clear compatibility date. Preserve the license. A public plugin is code other people will run, so make its actions easy to inspect.

First-party references

- Getting started: <https://claude.dev/blog/getting-started-with-claude-code-mods/>
- Plugin reference: <https://code.claude.com/docs/en/plugins-reference>
- Marketplaces: <https://code.claude.com/docs/en/plugin-marketplaces>
- Example source: <https://github.com/anthropics/claude-code/tree/main/mods>

Documentation was checked October 2, 2026. APIs can change; use the generated definitions for your installed host.

Terminal Pet

A little company while Claude works.

Adds a pixel crab above the prompt. It reacts when Claude reads, writes, waits, finishes, or hits an error. Its belly reflects conversation fullness.

1	2	3
Claude reads	Pet changes mood	You see activity

Terminal: install once. Add the marketplace using Start Here first. Choose one scope; run project commands from the target project. Start a fresh session.

```
claude plugin install terminal-pet@claude-mods-kit --scope user
claude plugin install terminal-pet@claude-mods-kit --scope project
```

Temporary trial from the kit folder: `bash scripts/try.sh terminal-pet`

Claude Code chat: Start with `/pet on`, then `/pet party` for an instant preview. Ask Claude to read the demo files to see real activity.

Controls below are a menu. Send one at a time.

```
/pet on
/pet name Pixel
/pet party
/pet wake
/pet off
```

Demo prompt: Read README.md and src/auth/login.ts. Explain what the demo does in two short sentences.

Plain English: Use the slash commands. This mod does not register a plain-English control tool.

Off / hide: `/pet off` To unload it for future sessions, use the terminal command below. Change user to project/local if needed.

```
claude plugin disable terminal-pet@claude-mods-kit --scope user
```

Know the limits: A mood is decoration. A nervous pet does not stop a dangerous action. Feed and party are preview controls, not real usage readings.

Terminal Pet • Creation prompt

Copy this whole specification into Claude Code in a new project.

Build a Claude Code mod packaged as a plugin. Start by reading the current official mods documentation and the generated types for my installed Claude Code version. Do not guess hook or UI APIs. Work in a new plugin folder, not my global configuration. Create a valid `.claude-plugin/plugin.json` and `hooks/hooks.json` pointing to an exported register function in an `.mjs` module. Use the sandbox host APIs, not unsupported Node or browser APIs. Preserve the normal hook chain with next where appropriate. Make settings explicit, keep work bounded, and keep permission prompts visible. Never collect telemetry or transmit my files to an unrelated service. Do not install or enable it globally without my request.

Build Terminal Pet: Draw a pixel crab above the prompt with distinct idle, reading, writing, working, waiting, error, sleeping and celebration states. Use original pixel art. Animate terminal raster and desktop SVG separately within host limits. Reflect measured context fullness in the belly; label forced preview states. Support name, on, off, sleep, wake, party and feed controls. Keep permission surveys visible. Persist preferences and stop needless redraws when hidden.

Acceptance checks: Test every sprite size, color budget, mood transition, hide/show, narrow layouts, desktop SVG limits and permission-survey precedence.

Deliver the plugin source, plain-English README, exact temporary-load and user/project install commands, slash commands, a realistic demo prompt, an off switch, limitations and tests. Run `claude plugin validate` and `claude plugin test`. Distinguish tests with fake events from actual terminal/desktop checks. If an API is unavailable, explain the gap instead of replacing it with pretend behavior. Show me the result in a throwaway demo folder. A creation prompt is a specification, not a promise of a perfect first build.

Coral Skin

Make the work easier to scan.

Changes supported Claude Code message and tool rows to a coral, ink and cream style. Colored chips help you spot reads, edits and commands.

1	2	3
A tool row renders	Skin adds styling	Work is easier to scan

Terminal: install once. Add the marketplace using Start Here first. Choose one scope; run project commands from the target project. Start a fresh session.

```
claude plugin install coral-skin@claude-mods-kit --scope user
claude plugin install coral-skin@claude-mods-kit --scope project
```

Temporary trial from the kit folder: `bash scripts/try.sh coral-skin`

Claude Code chat: Run `/skin on`, then ask Claude to inspect the demo. Run `/skin off` to compare.

Controls below are a menu. Send one at a time.

```
/skin on
/skin risk
/skin off
```

Demo prompt: Read README.md, inspect `src/auth/login.ts`, and explain the login flow without editing anything.

Plain English: Use `/skin`. Asking for a color in normal chat does not reliably change this mod.

Off / hide: `/skin off` To unload it for future sessions, use the terminal command below. Change user to project/local if needed.

```
claude plugin disable coral-skin@claude-mods-kit --scope user
```

Know the limits: Risk colors are visual hints, not security checks. This is Coral Skin; the separate Crimson Garden theme shown elsewhere is not bundled.

Coral Skin • Creation prompt

Copy this whole specification into Claude Code in a new project.

Build a Claude Code mod packaged as a plugin. Start by reading the current official mods documentation and the generated types for my installed Claude Code version. Do not guess hook or UI APIs. Work in a new plugin folder, not my global configuration. Create a valid `.claude-plugin/plugin.json` and `hooks/hooks.json` pointing to an exported register function in an `.mjs` module. Use the sandbox host APIs, not unsupported Node or browser APIs. Preserve the normal hook chain with next where appropriate. Make settings explicit, keep work bounded, and keep permission prompts visible. Never collect telemetry or transmit my files to an unrelated service. Do not install or enable it globally without my request.

Build Coral Skin: Style supported message, tool, spinner and footer components in coral, ink and cream. Show readable type-specific chips for reads, edits and shell commands. Keep original permission controls and tool results intact. Provide `/skin on`, `off` and `risk`. `Off` must restore normal rendering, including already mounted rows. Expose named palette constants so a beginner can ask to change the palette.

Acceptance checks: Test terminal and desktop component output, runtime toggling, risk-chip grouping, readable contrast and preservation of original permission UI.

Deliver the plugin source, plain-English README, exact temporary-load and user/project install commands, slash commands, a realistic demo prompt, an off switch, limitations and tests. Run `claude plugin validate` and `claude plugin test`. Distinguish tests with fake events from actual terminal/desktop checks. If an API is unavailable, explain the gap instead of replacing it with pretend behavior. Show me the result in a throwaway demo folder. A creation prompt is a specification, not a promise of a perfect first build.

Context Meter

See how full the conversation is.

Shows conversation fullness, session cost and available usage-limit readings above the prompt. Think of it as a dashboard gauge.

1	2	3
Usage is measured	Gauge updates	You see fullness

Terminal: install once. Add the marketplace using Start Here first. Choose one scope; run project commands from the target project. Start a fresh session.

```
claude plugin install context-meter@claude-mods-kit --scope user
claude plugin install context-meter@claude-mods-kit --scope project
```

Temporary trial from the kit folder: `bash scripts/try.sh context-meter`

Claude Code chat: It appears automatically after loading. Send a normal message and wait for a measurement. There is no `/context-meter` command.

Controls below are a menu. Send one at a time.

Automatic: no slash command.

Demo prompt: Read README.md and explain this project in plain English.

Plain English: Automatic. No special prompt is needed.

Off / hide: Disable the plugin and start a fresh session. To unload it for future sessions, use the terminal command below. Change user to project/local if needed.

```
claude plugin disable context-meter@claude-mods-kit --scope user
```

Know the limits: A full conversation is not an intelligence score. Dollars are API-equivalent usage, not a charge against your subscription. Missing usage data may be unavailable, not zero.

Context Meter · Creation prompt

Copy this whole specification into Claude Code in a new project.

Build a Claude Code mod packaged as a plugin. Start by reading the current official mods documentation and the generated types for my installed Claude Code version. Do not guess hook or UI APIs. Work in a new plugin folder, not my global configuration. Create a valid `.claude-plugin/plugin.json` and `hooks/hooks.json` pointing to an exported register function in an `.mjs` module. Use the sandbox host APIs, not unsupported Node or browser APIs. Preserve the normal hook chain with next where appropriate. Make settings explicit, keep work bounded, and keep permission prompts visible. Never collect telemetry or transmit my files to an unrelated service. Do not install or enable it globally without my request.

Build Context Meter: Render a compact AbovePrompt gauge using actual session context, cost and rate-limit data. Label percent fullness, token use, session cost and available five-hour/weekly limits. Handle missing data honestly. Respond to session measurement events. Yield to permission surveys. Keep it legible on narrow terminals and desktop. Do not invent a cost estimate when the host does not provide one.

Acceptance checks: Test changing measurements, missing or zero values, narrow rendering and permission surveys. Use no model call to draw the meter.

Deliver the plugin source, plain-English README, exact temporary-load and user/project install commands, slash commands, a realistic demo prompt, an off switch, limitations and tests. Run `claude plugin validate` and `claude plugin test`. Distinguish tests with fake events from actual terminal/desktop checks. If an API is unavailable, explain the gap instead of replacing it with pretend behavior. Show me the result in a throwaway demo folder. A creation prompt is a specification, not a promise of a perfect first build.

Repo Heatmap

See which files Claude touches.

Draws a map of files in the current project. File areas reflect size, and colors react to reads, searches, edits and failed operations.

1	2	3
A file is touched	Map colors change	You see the path

Terminal: install once. Add the marketplace using Start Here first. Choose one scope; run project commands from the target project. Start a fresh session.

```
claude plugin install repo-heatmap@claude-mods-kit --scope user
claude plugin install repo-heatmap@claude-mods-kit --scope project
```

Temporary trial from the kit folder: `bash scripts/try.sh repo-heatmap`

Claude Code chat: In the demo project, open `/heatmap` before sending the prompt. Watch files light up while Claude works.

Controls below are a menu. Send one at a time.

```
/heatmap open
/heatmap size lines
/heatmap rescan
/heatmap reset
/heatmap close
```

Demo prompt: Investigate the login flow, from `src/ui/LoginForm.tsx` through `src/auth/login.ts`, `session.ts` and `tokens.ts`. Read the files and explain how a user gets signed in. Do not edit anything.

Plain English: Open it with `/heatmap`. The investigation itself is a normal English prompt.

Off / hide: `/heatmap close` To unload it for future sessions, use the terminal command below. Change user to `project/local` if needed.

```
claude plugin disable repo-heatmap@claude-mods-kit --scope user
```

Know the limits: Tracks activity observed during this session. A bright file is not proof of importance or a bug. Some shell activity is inferred and may not map perfectly.

Repo Heatmap · Creation prompt

Copy this whole specification into Claude Code in a new project.

Build a Claude Code mod packaged as a plugin. Start by reading the current official mods documentation and the generated types for my installed Claude Code version. Do not guess hook or UI APIs. Work in a new plugin folder, not my global configuration. Create a valid `.claude-plugin/plugin.json` and `hooks/hooks.json` pointing to an exported register function in an `.mjs` module. Use the sandbox host APIs, not unsupported Node or browser APIs. Preserve the normal hook chain with next where appropriate. Make settings explicit, keep work bounded, and keep permission prompts visible. Never collect telemetry or transmit my files to an unrelated service. Do not install or enable it globally without my request.

Build Repo Heatmap: Scan the working project and draw a squarified treemap with file area by bytes or lines. Observe direct file tools and recognizable shell/search results, distinguishing reads, edits and failures. Animate touches, then settle to a visited tint. Ignore generated and hidden bulk. Cap rendering work for large projects. Provide open, close, reset, rescan and size commands. Explain an empty or too-narrow pane.

Acceptance checks: Test nonoverlap and proportional areas, path normalization, search hits, failure classification, bounded color count, animation decay and terminal/desktop rendering.

Deliver the plugin source, plain-English README, exact temporary-load and user/project install commands, slash commands, a realistic demo prompt, an off switch, limitations and tests. Run `claude plugin validate` and `claude plugin test`. Distinguish tests with fake events from actual terminal/desktop checks. If an API is unavailable, explain the gap instead of replacing it with pretend behavior. Show me the result in a throwaway demo folder. A creation prompt is a specification, not a promise of a perfect first build.

Flight Recorder

See the work unfold on a timeline.

Shows turns, tool activity and helper activity over time so you can see when work happened and what ran alongside it.

1	2	3
Work starts / ends	Timeline records it	You see the sequence

Terminal: install once. Add the marketplace using Start Here first. Choose one scope; run project commands from the target project. Start a fresh session.

```
claude plugin install flight-recorder@claude-mods-kit --scope user
claude plugin install flight-recorder@claude-mods-kit --scope project
```

Temporary trial from the kit folder: `bash scripts/try.sh flight-recorder`

Claude Code chat: Run `/timeline open`, then ask for two read-only helper investigations. Helpers appear only if Claude actually starts them.

Controls below are a menu. Send one at a time.

```
/timeline open
/timeline turns 5
/timeline clear
/timeline close
```

Demo prompt: Use two read-only helper agents: one to inspect the login flow, and one to inspect configuration and tests. Wait for both, then give me a short combined summary. Do not edit files.

Plain English: Use `/timeline` to open it. Request helper work in plain English.

Off / hide: `/timeline close` To unload it for future sessions, use the terminal command below. Change user to `project/local` if needed.

```
claude plugin disable flight-recorder@claude-mods-kit --scope user
```

Know the limits: A timeline is a record of observed events, not proof that parallel work was faster. Cost figures use a static estimate and can differ from billing, including cache-write charges.

Flight Recorder · Creation prompt

Copy this whole specification into Claude Code in a new project.

Build a Claude Code mod packaged as a plugin. Start by reading the current official mods documentation and the generated types for my installed Claude Code version. Do not guess hook or UI APIs. Work in a new plugin folder, not my global configuration. Create a valid `.claude-plugin/plugin.json` and `hooks/hooks.json` pointing to an exported register function in an `.mjs` module. Use the sandbox host APIs, not unsupported Node or browser APIs. Preserve the normal hook chain with next where appropriate. Make settings explicit, keep work bounded, and keep permission prompts visible. Never collect telemetry or transmit my files to an unrelated service. Do not install or enable it globally without my request.

Build Flight Recorder: Build a timeline pane for main turns, tool spans and named helper lanes. Track starts and completions, label failures and elapsed times, and keep bounded history. Do not count background notifications as fresh human tasks. Support open, close, clear and turns. Preserve the underlying work. If costs are shown, label them estimates and show missing model prices as unknown.

Acceptance checks: Test overlapping helpers, parent/child events, incomplete spans, notification filtering, bounded history and both UI surfaces. Do not claim a speedup or cost saving from fixture data.

Deliver the plugin source, plain-English README, exact temporary-load and user/project install commands, slash commands, a realistic demo prompt, an off switch, limitations and tests. Run `claude plugin validate` and `claude plugin test`. Distinguish tests with fake events from actual terminal/desktop checks. If an API is unavailable, explain the gap instead of replacing it with pretend behavior. Show me the result in a throwaway demo folder. A creation prompt is a specification, not a promise of a perfect first build.

Model Router

Choose a lighter model for helper work.

Routes supported helper-agent steps to Sonnet by default while the main conversation keeps its own model. Haiku is an option.

1	2	3
A helper takes a step	Model choice is checked	Eligible work is routed

Terminal: install once. Add the marketplace using Start Here first. Choose one scope; run project commands from the target project. Start a fresh session.

```
claude plugin install model-router@claude-mods-kit --scope user
claude plugin install model-router@claude-mods-kit --scope project
```

Temporary trial from the kit folder: `bash scripts/try.sh model-router`

Claude Code chat: Check `/router`, choose subagents mode, then request a helper. A route only happens if the pricing table knows the original model and considers the target cheaper.

Controls below are a menu. Send one at a time.

```
/router on
/router mode subagents
/router sonnet
/router
/router off
```

Demo prompt: Use the Explore agent to find where the port is configured in this folder and what reads `config/config.json`, then tell me in two sentences. Do not edit files.

Plain English: Controls use `/router`. Ask Claude to use a helper in a normal prompt.

Off / hide: `/router off` To unload it for future sessions, use the terminal command below. Change user to `project/local` if needed.

```
claude plugin disable model-router@claude-mods-kit --scope user
```

Know the limits: Changes which model does real work. Review the answer. Model availability and prices can change; unknown models may stay unchanged. Savings are estimates, not a quality guarantee. Experimental steps/both modes can lose cache benefits.

Model Router · Creation prompt

Copy this whole specification into Claude Code in a new project.

Build a Claude Code mod packaged as a plugin. Start by reading the current official mods documentation and the generated types for my installed Claude Code version. Do not guess hook or UI APIs. Work in a new plugin folder, not my global configuration. Create a valid `.claude-plugin/plugin.json` and `hooks/hooks.json` pointing to an exported register function in an `.mjs` module. Use the sandbox host APIs, not unsupported Node or browser APIs. Preserve the normal hook chain with next where appropriate. Make settings explicit, keep work bounded, and keep permission prompts visible. Never collect telemetry or transmit my files to an unrelated service. Do not install or enable it globally without my request.

Build Model Router: Default to routing only helper-agent steps to Sonnet, with Haiku selectable. Keep every main-conversation step on its own model in this mode. Treat both undefined and null agent IDs as the main conversation. Route only known more-expensive models. Keep unknown models unchanged. Support on, off, status and explicit experimental steps/both modes. Persist preferences. Report decisions and clearly labeled price-table estimates without guaranteeing savings or quality.

Acceptance checks: Test null/undefined main IDs, actual helper IDs, cheaper and unknown source models, disabled mode, model settings and cold/warm-cache accounting. Never invent model IDs or prices; inspect current supported ones.

Deliver the plugin source, plain-English README, exact temporary-load and user/project install commands, slash commands, a realistic demo prompt, an off switch, limitations and tests. Run `claude plugin validate` and `claude plugin test`. Distinguish tests with fake events from actual terminal/desktop checks. If an API is unavailable, explain the gap instead of replacing it with pretend behavior. Show me the result in a throwaway demo folder. A creation prompt is a specification, not a promise of a perfect first build.

Output Tray

Find the files Claude just made.

Collects new files created during the session in a panel with copy, open and reveal actions.

1	2	3
A new file is made	Tray adds a row	You choose Open

Terminal: install once. Add the marketplace using Start Here first. Choose one scope; run project commands from the target project. Start a fresh session.

```
claude plugin install output-tray@claude-mods-kit --scope user
claude plugin install output-tray@claude-mods-kit --scope project
```

Temporary trial from the kit folder: `bash scripts/try.sh output-tray`

Claude Code chat: Run `/tray show`, send the file-creation prompt, then `/tray list`. This starter kit implements Open and Reveal using macOS Finder.

Controls below are a menu. Send one at a time.

```
/tray show
/tray list
/tray open 1
/tray reveal 1
/tray copy 1
/tray close
```

Demo prompt: Create demo-output if it does not exist. Write a short explanation of this demo project to demo-output/project-overview.md and a file inventory to demo-output/file-inventory.csv. Use a new numbered filename if either exists. Use the Write tool for each file. Do not edit existing files.

Plain English: “Open the output tray” or “List the files in the output tray” is supported by a registered tool. The model cannot use that tool to launch files.

Off / hide: `/tray close` To unload it for future sessions, use the terminal command below. Change user to project/local if needed.

```
claude plugin disable output-tray@claude-mods-kit --scope user
```

Know the limits: Lists newly created files, not every edited file. Start tracking before the work. Clear removes the list, not the files. Mac Open/Reveal is not a cross-platform file opener.

Output Tray · Creation prompt

Copy this whole specification into Claude Code in a new project.

Build a Claude Code mod packaged as a plugin. Start by reading the current official mods documentation and the generated types for my installed Claude Code version. Do not guess hook or UI APIs. Work in a new plugin folder, not my global configuration. Create a valid `.claude-plugin/plugin.json` and `hooks/hooks.json` pointing to an exported register function in an `.mjs` module. Use the sandbox host APIs, not unsupported Node or browser APIs. Preserve the normal hook chain with next where appropriate. Make settings explicit, keep work bounded, and keep permission prompts visible. Never collect telemetry or transmit my files to an unrelated service. Do not install or enable it globally without my request.

Build Output Tray: Track new files created by direct Write, notebook edits, shell commands and turn-boundary snapshots. Ignore failed, denied or staged-only writes. Deduplicate paths and flag missing files. Provide a list pane, number selection, copy path, explicit Open and Reveal. Scripts must open as text and apps/executables must only be revealed. Never execute a file automatically. Let the model show/list the tray but never open a file. Handle null main-agent IDs.

Acceptance checks: Test direct and shell creation, existing files, denial, deletion, spaced paths, helper tags, null turn boundaries, narrow-pane fallback and exact user-only open arguments.

Deliver the plugin source, plain-English README, exact temporary-load and user/project install commands, slash commands, a realistic demo prompt, an off switch, limitations and tests. Run `claude plugin validate` and `claude plugin test`. Distinguish tests with fake events from actual terminal/desktop checks. If an API is unavailable, explain the gap instead of replacing it with pretend behavior. Show me the result in a throwaway demo folder. A creation prompt is a specification, not a promise of a perfect first build.

Changes Receipt

Get a clear list of what changed.

Summarizes files created, edited and deleted during a tracked turn. It separates successful changes from failed attempts.

1	2	3
A turn finishes	Changes are compared	You get a receipt

Terminal: install once. Add the marketplace using Start Here first. Choose one scope; run project commands from the target project. Start a fresh session.

```
claude plugin install changes-receipt@claude-mods-kit --scope user
claude plugin install changes-receipt@claude-mods-kit --scope project
```

Temporary trial from the kit folder: `bash scripts/try.sh changes-receipt`

Claude Code chat: Run `/receipt` on before the file-creation prompt. Wait until Claude finishes, then run `/receipt last`.

Controls below are a menu. Send one at a time.

```
/receipt on
/receipt open
/receipt last
/receipt close
/receipt off
```

Demo prompt: Create demo-output if needed. Use the Write tool to create demo-output/receipt-example.md with three bullet points about the login flow. If that filename exists, create a new numbered filename. Do not modify any other file.

Plain English: “Show the changes receipt” is supported. Slash commands are the clearest way to check the last tracked turn.

Off / hide: `/receipt off` To unload it for future sessions, use the terminal command below. Change user to project/local if needed.

```
claude plugin disable changes-receipt@claude-mods-kit --scope user
```

Know the limits: A receipt is not undo, backup or perfect attribution. Other programs changing files at the same time can confuse snapshots. It does not backfill old sessions.

Changes Receipt · Creation prompt

Copy this whole specification into Claude Code in a new project.

Build a Claude Code mod packaged as a plugin. Start by reading the current official mods documentation and the generated types for my installed Claude Code version. Do not guess hook or UI APIs. Work in a new plugin folder, not my global configuration. Create a valid `.claude-plugin/plugin.json` and `hooks/hooks.json` pointing to an exported register function in an `.mjs` module. Use the sandbox host APIs, not unsupported Node or browser APIs. Preserve the normal hook chain with next where appropriate. Make settings explicit, keep work bounded, and keep permission prompts visible. Never collect telemetry or transmit my files to an unrelated service. Do not install or enable it globally without my request.

Build Changes Receipt: Snapshot the project at main-turn boundaries and combine Git changes or a filesystem fallback with direct tool evidence. Report creates, changes, deletions, renames and failed attempts separately. Preserve writes from the main turn when helpers start and finish. Accept null main-agent IDs. Provide on, off, open, close, last and bounded history. Add a natural-language tool that reads receipts. Do not imply rollback.

Acceptance checks: Test dirty files before a turn, changes to dirty files, failed and denied writes, shell renames, non-Git projects, outside paths, helper interleaving, missing turn-start evidence and both UI surfaces.

Deliver the plugin source, plain-English README, exact temporary-load and user/project install commands, slash commands, a realistic demo prompt, an off switch, limitations and tests. Run `claude plugin validate` and `claude plugin test`. Distinguish tests with fake events from actual terminal/desktop checks. If an API is unavailable, explain the gap instead of replacing it with pretend behavior. Show me the result in a throwaway demo folder. A creation prompt is a specification, not a promise of a perfect first build.

Session Bookmarks

Leave yourself a useful “come back here” note.

Saves a title and a short note with the project and session reference, so you can find the conversation again.

1	2	3
You save a note	A bookmark is stored	You copy Resume

Terminal: install once. Add the marketplace using Start Here first. Choose one scope; run project commands from the target project. Start a fresh session.

```
claude plugin install session-bookmarks@claude-mods-kit --scope user
claude plugin install session-bookmarks@claude-mods-kit --scope project
```

Temporary trial from the kit folder: `bash scripts/try.sh session-bookmarks`

Claude Code chat: Run `/bm save Login walkthrough`; next run `/bm list`. Use `/bm resume 1` and paste the copied command into a new terminal.

Controls below are a menu. Send one at a time.

```
/bm save Login walkthrough
/bm list
/bm find login
/bm open
/bm resume 1
/bm close
```

Demo prompt: Bookmark this session as “Login walkthrough” with the note “Read the auth files; next step is writing a plain-English overview.”

Plain English: “Bookmark this session” and “Find my bookmark about login” are supported.

Off / hide: `/bm close` To unload it for future sessions, use the terminal command below. Change user to `project/local` if needed.

```
claude plugin disable session-bookmarks@claude-mods-kit --scope user
```

Know the limits: Resume copies a terminal command; it does not switch your current desktop chat. The folder and original transcript must still exist. A blank `/bm` uses a short model call to suggest a title; an explicit note avoids that call.

Session Bookmarks · Creation prompt

Copy this whole specification into Claude Code in a new project.

Build a Claude Code mod packaged as a plugin. Start by reading the current official mods documentation and the generated types for my installed Claude Code version. Do not guess hook or UI APIs. Work in a new plugin folder, not my global configuration. Create a valid `.claude-plugin/plugin.json` and `hooks/hooks.json` pointing to an exported register function in an `.mjs` module. Use the sandbox host APIs, not unsupported Node or browser APIs. Preserve the normal hook chain with next where appropriate. Make settings explicit, keep work bounded, and keep permission prompts visible. Never collect telemetry or transmit my files to an unrelated service. Do not install or enable it globally without my request.

Build Session Bookmarks: Save only explicit bookmarks with title, note, project, branch, working folder, session ID, transcript path and timestamps. Store a user-local JSON file with restrictive permissions and atomic writes. Do not copy transcripts. Allow explicit notes without model calls; optional automatic titles use a bounded call with fallback. Implement list, search, rename, delete, panel and resume. Resume must safely quote the folder and copy/print a command, never claim in-chat switching.

Acceptance checks: Test persistence, corrupt-file backup, missing folders, quoted paths, explicit-note no-call behavior, title failure fallback, confirm-before-delete in the pane and both UI surfaces.

Deliver the plugin source, plain-English README, exact temporary-load and user/project install commands, slash commands, a realistic demo prompt, an off switch, limitations and tests. Run `claude plugin validate` and `claude plugin test`. Distinguish tests with fake events from actual terminal/desktop checks. If an API is unavailable, explain the gap instead of replacing it with pretend behavior. Show me the result in a throwaway demo folder. A creation prompt is a specification, not a promise of a perfect first build.

Auto Handoff

Leave the next chat a useful starting point.

Writes a summary of progress, decisions and next steps into the project so a fresh conversation can continue with that summary.

1	2	3
You save / limit nears	AI writes a summary	A new chat reads it

Terminal: install once. Add the marketplace using Start Here first. Choose one scope; run project commands from the target project. Start a fresh session.

```
claude plugin install auto-handoff@claude-mods-kit --scope user
claude plugin install auto-handoff@claude-mods-kit --scope project
```

Temporary trial from the kit folder: `bash scripts/try.sh auto-handoff`

Claude Code chat: After a short task, run `/autohandoff` now. Read the generated handoff before trusting it. In a fresh session in the same folder, ask Claude to read `handoff/LATEST.md`.

Controls below are a menu. Send one at a time.

```
/autohandoff
/autohandoff now
/autohandoff threshold 85
/autohandoff resume
/autohandoff off
```

Demo prompt: Read `handoff/LATEST.md`, follow its link to the handoff file, and summarize the next step. Do not change anything until I choose a task.

Plain English: The save controls use `/autohandoff`. Reading the handoff in the next chat is a normal prompt.

Off / hide: `/autohandoff off` To unload it for future sessions, use the terminal command below. Change user to `project/local` if needed.

```
claude plugin disable auto-handoff@claude-mods-kit --scope user
```

Know the limits: Summaries can miss details and use an extra model call. The default automatic threshold is 85%. Files remain in your project after disabling; review them before sharing. It does not automatically create a new chat.

Auto Handoff · Creation prompt

Copy this whole specification into Claude Code in a new project.

Build a Claude Code mod packaged as a plugin. Start by reading the current official mods documentation and the generated types for my installed Claude Code version. Do not guess hook or UI APIs. Work in a new plugin folder, not my global configuration. Create a valid `.claude-plugin/plugin.json` and `hooks/hooks.json` pointing to an exported register function in an `.mjs` module. Use the sandbox host APIs, not unsupported Node or browser APIs. Preserve the normal hook chain with next where appropriate. Make settings explicit, keep work bounded, and keep permission prompts visible. Never collect telemetry or transmit my files to an unrelated service. Do not install or enable it globally without my request.

Build Auto Handoff: At a configurable context threshold, request a concise handoff summary and write a timestamped Markdown file plus `LATEST.md`. Include goals, completed work, decisions, touched files, checks and next actions; tell the summarizer to exclude secrets. Preserve existing project pointers and add handoff to Git ignore without rewriting unrelated entries. Provide status, now, resume, threshold, on and off. Do not overwrite a good handoff with an empty reply. Resume validates the target before submitting a prompt.

Acceptance checks: Test threshold crossing/rearming, before-compaction behavior, failed-save retry, empty-summary protection, missing-target resume, file names, ignore preservation and extra-model-call boundaries.

Deliver the plugin source, plain-English README, exact temporary-load and user/project install commands, slash commands, a realistic demo prompt, an off switch, limitations and tests. Run `claude plugin validate` and `claude plugin test`. Distinguish tests with fake events from actual terminal/desktop checks. If an API is unavailable, explain the gap instead of replacing it with pretend behavior. Show me the result in a throwaway demo folder. A creation prompt is a specification, not a promise of a perfect first build.

Your data and the off switch

What this kit contains

Plugin source, synthetic tests, a fictional demo project, creation prompts and guides. It contains no creator settings, keys, chat history, real bookmarks or private work files. There is no added analytics or unrelated network service in the plugin code. Claude Code itself still follows your account and workspace permissions.

What the mods can read or write

Pet, Skin and Meter observe session events and usage. Heatmap, Timeline, Tray and Receipt observe project activity; some read file metadata or content and run local Git commands. This is real code running with the host capabilities available to it.

Router changes supported model requests. Bookmark auto-titling and Handoff summaries make extra model calls through Claude Code. Do not assume that “mod” means offline or zero usage.

Bookmarks save titles, notes, folder paths and session references to `~/Claude/bookmarks/bookmarks.json` unless overridden. They do not copy the transcript, but a note can still contain sensitive text. `/bm delete N` removes a chosen bookmark. Export a backup first if you need it.

Handoff writes timestamped summaries under the current project's `handoff/` folder, plus `LATEST.md` and sometimes `ACTIVE_PROJECT.md`, and adds an ignore entry. A summary can contain project details even when asked to avoid secrets. Read it before sharing. Removing the plugin does not delete these project files.

The temporary demo helper redirects bookmark storage into its own throwaway project. Normal installations use your normal bookmark location. Persistent preferences may live in the host's plugin store.

Hide, disable, uninstall

Hiding a panel does not necessarily stop tracking. Commands such as `/pet off`, `/router off` and `/receipt off` control that mod's feature. Disabling unloads the plugin in future sessions. Uninstalling removes its installation in the chosen scope. None of these retroactively changes completed model calls.

From the kit root, choose one:

```
bash scripts/manage.sh disable user all
bash scripts/manage.sh uninstall user all
```

For project/local installs run the script from the actual project folder, using the script's full path:

```
bash /path/to/kit/scripts/manage.sh disable project all
bash /path/to/kit/scripts/manage.sh disable local all
```

`/path/to/kit` is a placeholder. Repeat for every project where you installed the kit. There is no safe one-command scan of every folder on your computer. The script reports failures rather than claiming

everything is off. Close existing sessions, including any started with `--plugin-dir`.

The uninstall helper uses `--keep-data` to avoid silently deleting saved plugin preferences. Review and delete remaining data separately if you want it gone. Remove only the bookmark entries or demo handoffs you intend to remove, not an entire folder that may hold useful work. Do not remove `~/ .claude` or unrelated marketplaces.

After uninstalling all kit plugins, you may remove the marketplace through Claude Code's plugin marketplace controls. Keep the downloaded kit as a backup or delete just that extracted folder when you no longer need it.

What was checked

Test host: macOS, Claude Code CLI 2.1.287. Release: 1.0.0.

All ten plugin manifests, the marketplace and the teaching template validate. The ten plugins pass 123 unit tests; the template passes one additional test, for 124 passing tests and zero failures. These are development checks using the Claude Code test harness. They are not held-out performance measurements or proof of savings, quality or live desktop parity.

The helper scripts were exercised with a fake Claude binary: all-ten and single-plugin selection, explicit scopes, project paths with spaces, keep-data uninstall, rejection of invalid input, unique demo folders and isolated demo bookmark storage. That check did not modify the test machine's actual plugin settings.

The PDF was rendered and visually inspected. The offline guide was checked at desktop and mobile widths. Public files were scanned for personal paths, creator names, credentials and session exports. Only synthetic demo data is included. All authored content and scripts are inspectable in this repository.

Remaining limits: no fresh live ten-mod desktop session was certified in this packaging run. No Windows/Linux live test, subscription-billing reconciliation, external checkout or downstream customer automation test was performed. Output Tray Open/Reveal currently uses macOS. Unknown model IDs can prevent routing. Host APIs can change.

Two distribution-only regression fixes handle null main-agent IDs in Model Router and Output Tray. Original installed plugins were not changed. See CHANGELOG.md. The guide deliberately keeps these limits visible beside the relevant feature.